

JC4004 – Computational Intelligence

Sessions 29 & 30: Attention and transformers

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

Oct-Dec 2025

Background of attention

- Traditionally, LSTM was used for sequence-to-sequence conversion tasks, such as machine translation
 - However, LSTM processes data in a strictly sequential order, and information about the past states is only preserved via the hidden states
 - LSTM struggles in cases when the output token depends highly on the input token that appeared in the input sequence a long time earlier
- Attention mechanism aims to address the weaknesses of LSTM
 - Attention mechanism treats sequences as a whole and allows direct access to all previous states and tokens when generating output tokens

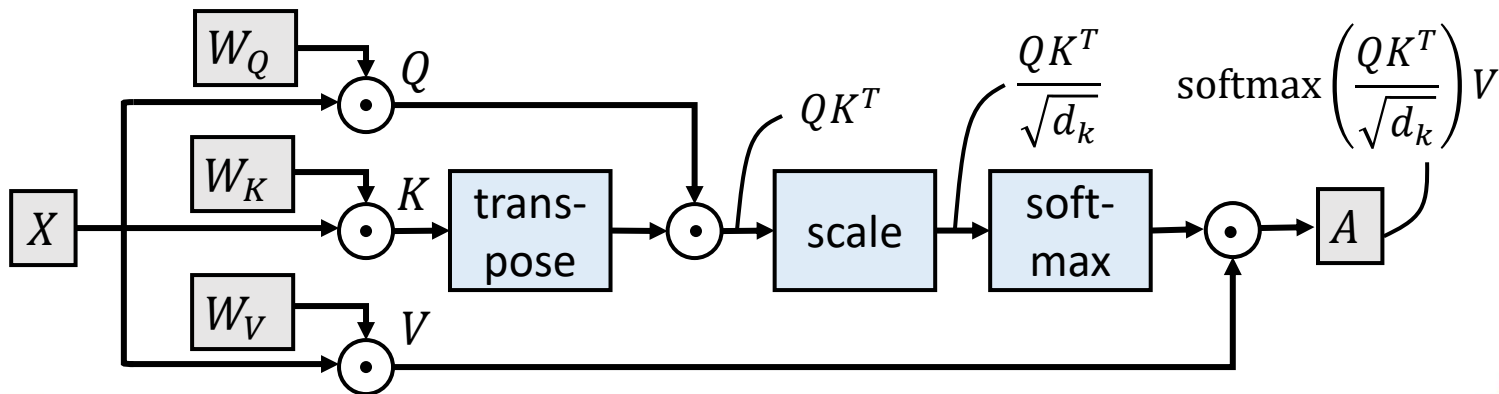
Scaled dot-product attention

- Several different attention mechanisms have been proposed
- Modern transformer architectures are based on *scaled dot-product attention* from paper “*Attention Is All You Need*” by **Vaswani** et al. in 2017
 - Attention A formulated as a function of *query*, *key* and *value* matrices Q , K , and V , with parameter d_k indicating the length of keys:

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

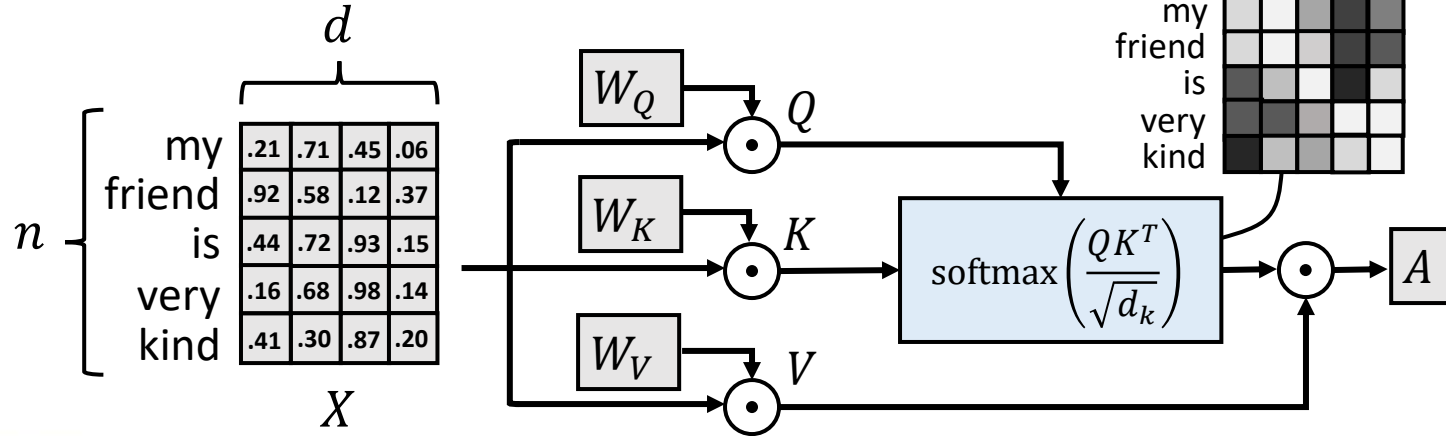
Self-attention

- *Self-attention* represents the input sequence weighted by the mutual relationships (attention) between the n input elements
 - Matrices Q , K , and V are computed from the input matrix X by multiplying with learnable weight matrices: $Q = W_Q X$, $K = W_K X$, and $V = W_V X$



Self-attention example

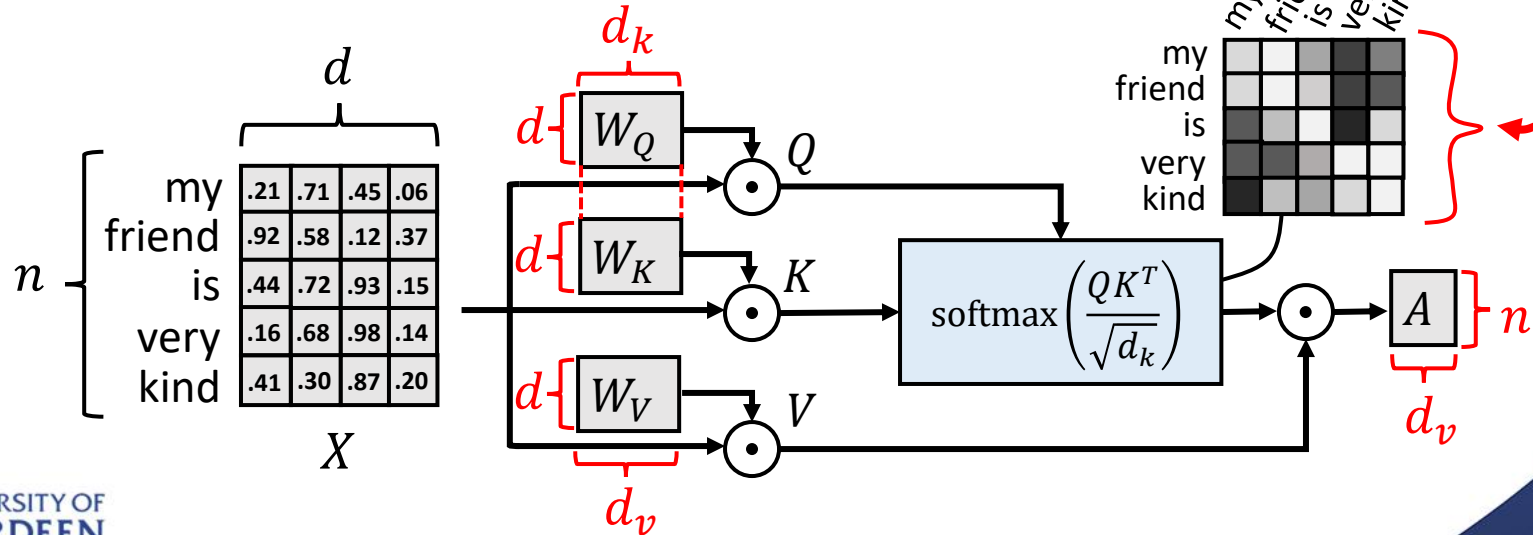
- In input X , each row represents an input token (e.g., a word in a sentence), coded as a vector of d values (d is the embedding size)
- Attention matrix relates tokens to each other



Self-attention example

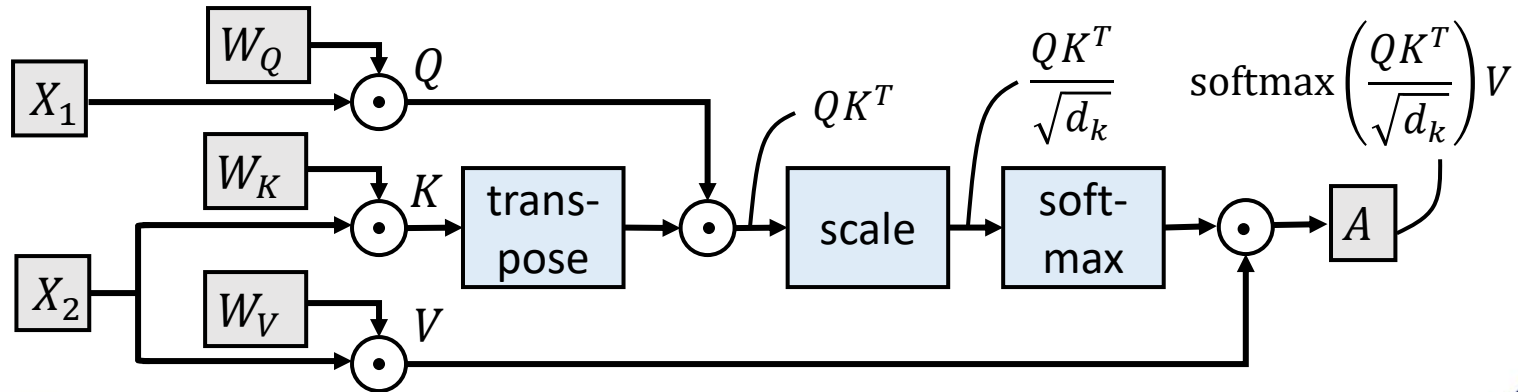
- In input X , each row represents an input token coded as a vector of d values (d is the embedding dimension)
- Attention matrix relates tokens to each other

Due to the softmax operation, sum of each row is one



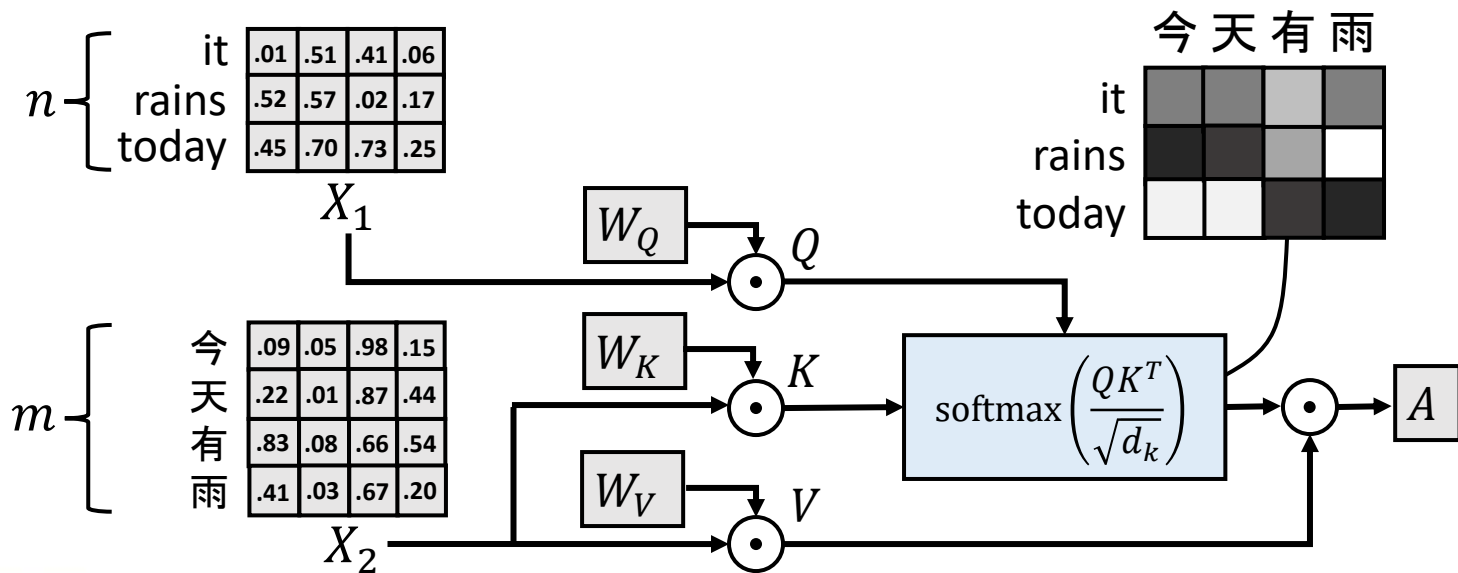
Cross-attention

- *Cross-attention* is similar to self-attention, except that two matrices X_1 and X_2 are used as input
 - Matrix Q , is computed from X_1 and matrices K and V from X_2
 - Number of tokens can be different, but embedding size must be the same



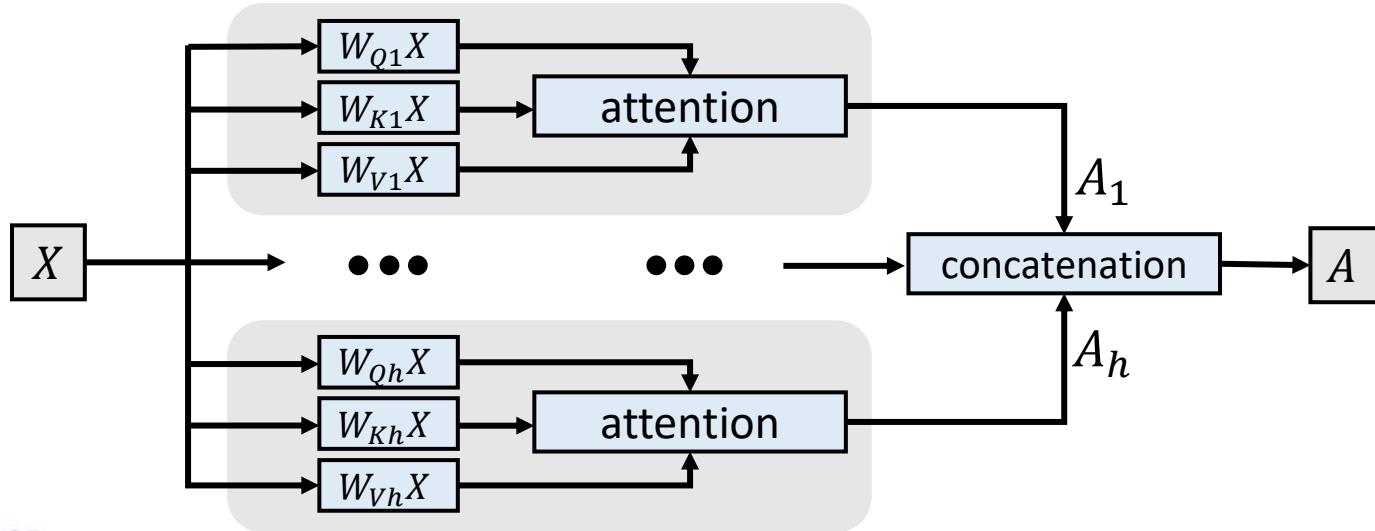
Cross-attention example

- Attention matrix evaluates the relationships between the tokens in X_1 and X_2



Multi-head attention

- In *multi-head attention (MHA)*, several versions of the weight matrices are used in parallel, and the final output is formed by concatenating

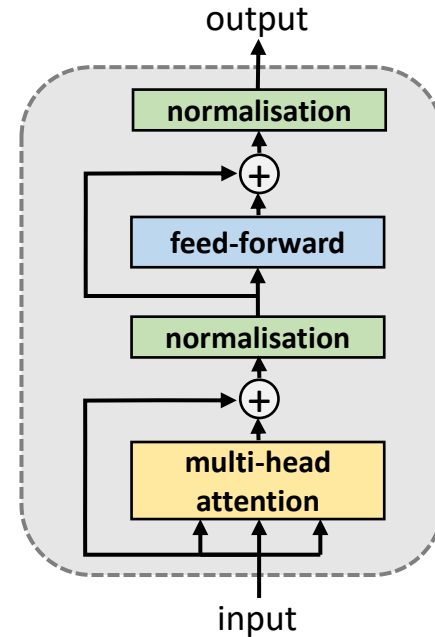


Transformers

- In practical applications, attention is often used as a building block
 - Attention matrix weights can be trained using backpropagation
 - *Transformers* are a family of network architectures using attention
- Initially, transformers were developed for sequence-to-sequence translation models such as machine translation:
 - *Transformer encoder* encodes input sequence to internal representation
 - *Transformer decoder* decodes the internal representation to target output
- Nowadays, transformers have many other applications as well

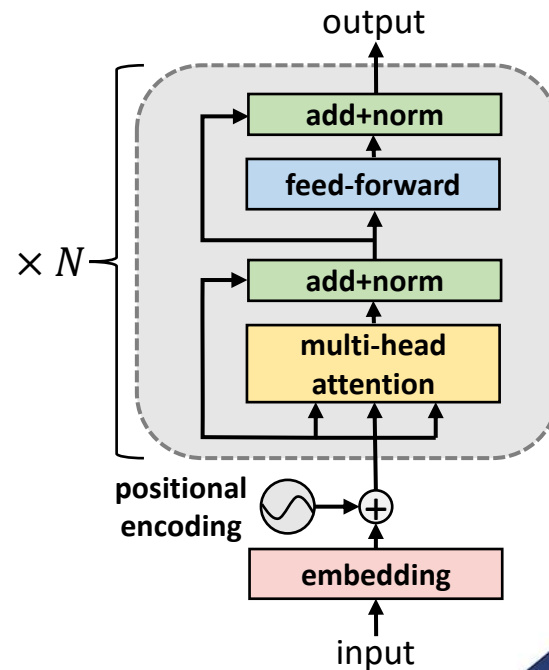
Transformer block

- A commonly used transformer block consists of an MHA layer, feed-forward layer, and skip connections
 - Input and output dimensions are the same
 - Normalisation layers (batch normalisation) often used after MHA and fully connected layers to avoid vanishing gradient problem
 - Skip connections used to bypass MHA and feed-forward layers



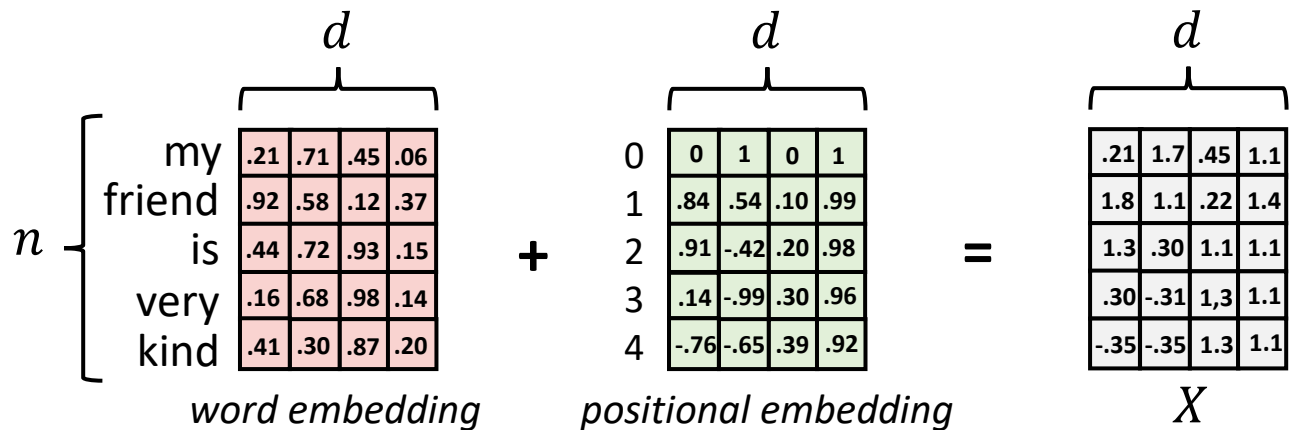
Transformer encoder

- A commonly used transformer encoder includes embedding, positional encoding, and one or multiple transformer blocks
 - Embedding converts input symbols (like words) into vector representations (tokens)
 - Positional encoding adds information about each token's position in the input sequence
 - Several transformer blocks can be stacked to make the full transformer encoder



Positional encoding

- Positional coding adds information about each token's position
 - Unique embedding for each position index in the sequence
 - Position embeddings are simply added to the original embeddings



Sinusoidal positional encoding

- Every position must have a unique encoding, and we cannot use the position indices directly, because the values would grow very large
- Sinusoidal positional encoding commonly used
 - Given embedding length d , encoding PE for position p computed as:

$$PE(p, i) = \begin{cases} \sin\left(\frac{p}{n^{i/d}}\right) & \text{if } i = 0, 2, 4, \dots \\ \cos\left(\frac{p}{n^{(i-1)/d}}\right) & \text{if } i = 1, 3, 5, \dots \end{cases}$$

- Value of n can be any positive integer, but 10,000 is recommended

Sinusoidal positional encoding example

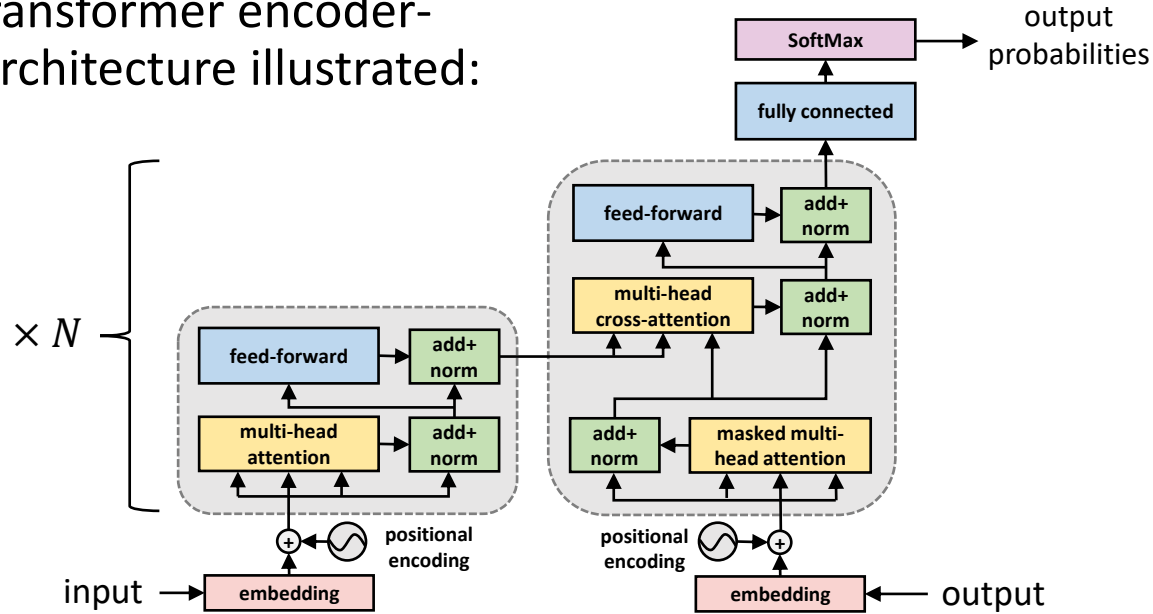
- Given embedding length $d = 4$ and choosing $n = 10,000$, we can compute the positional encodings for the three first indices as:

<i>position index</i>	$i = 0$	$i = 1$	$i = 2$	$i = 3$
0	$\sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0)$ <u>≈ 0</u>	$\cos\left(\frac{0}{10000^{0/4}}\right) = \cos(0)$ <u>≈ 1</u>	$\sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0)$ <u>≈ 0</u>	$\cos\left(\frac{0}{10000^{2/4}}\right) = \cos(0)$ <u>≈ 1</u>
1	$\sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1)$ <u>≈ 0.8415</u>	$\cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1)$ <u>≈ 0.5403</u>	$\sin\left(\frac{1}{10000^{2/4}}\right) = \sin\left(\frac{1}{100}\right)$ <u>≈ 0.0100</u>	$\cos\left(\frac{1}{10000^{2/4}}\right) = \cos\left(\frac{1}{100}\right)$ <u>≈ 1.0000</u>
2	$\sin\left(\frac{2}{10000^{0/4}}\right) = \sin(2)$ <u>≈ 0.9093</u>	$\cos\left(\frac{2}{10000^{0/4}}\right) = \cos(2)$ <u>≈ -0.4161</u>	$\sin\left(\frac{2}{10000^{2/4}}\right) = \sin\left(\frac{2}{100}\right)$ <u>≈ 0.0200</u>	$\cos\left(\frac{2}{10000^{2/4}}\right) = \cos\left(\frac{2}{100}\right)$ <u>≈ 0.9998</u>

Break: any questions?

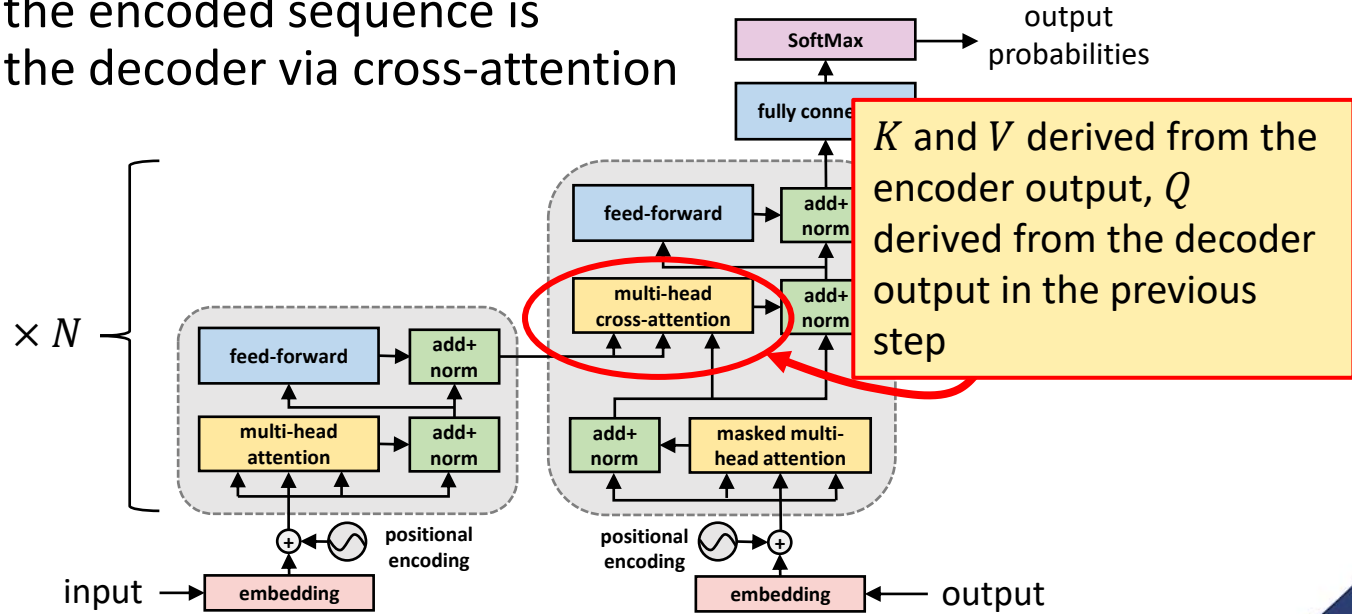
Transformer encoder-decoder model

- Original transformer encoder-decoder architecture illustrated:



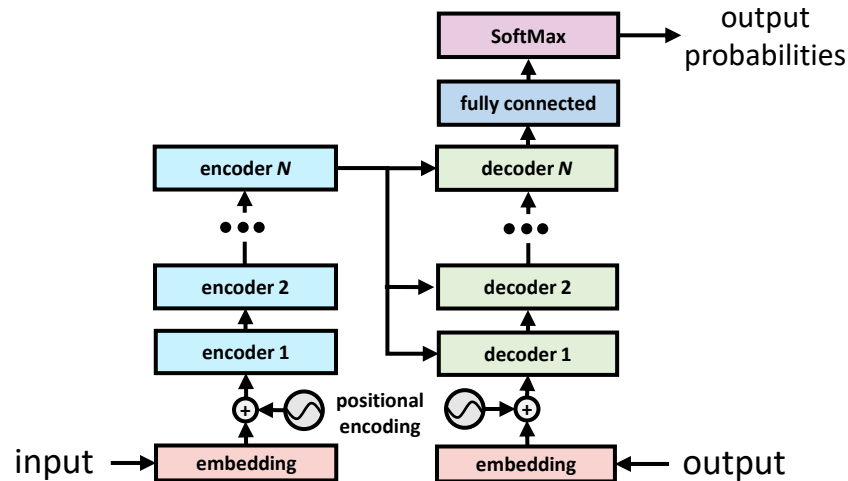
Transformer encoder-decoder model

- Note that the encoded sequence is passed to the decoder via cross-attention



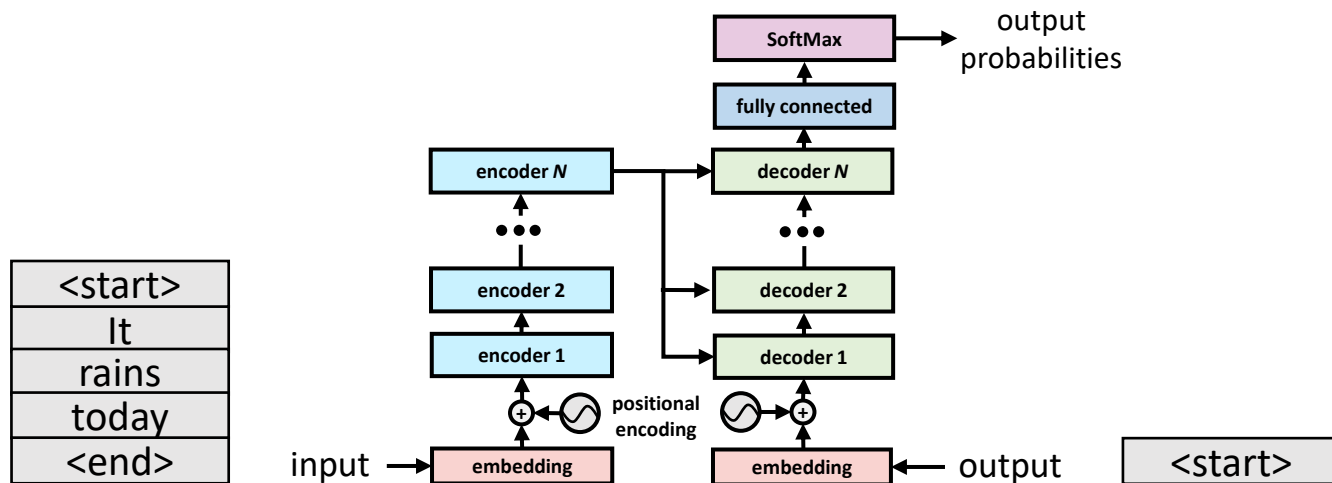
Transformer encoder-decoder in practice

- Practical encoder-decoders stack several encoders and decoders
 - In the original paper, six encoder and decoder blocks were used



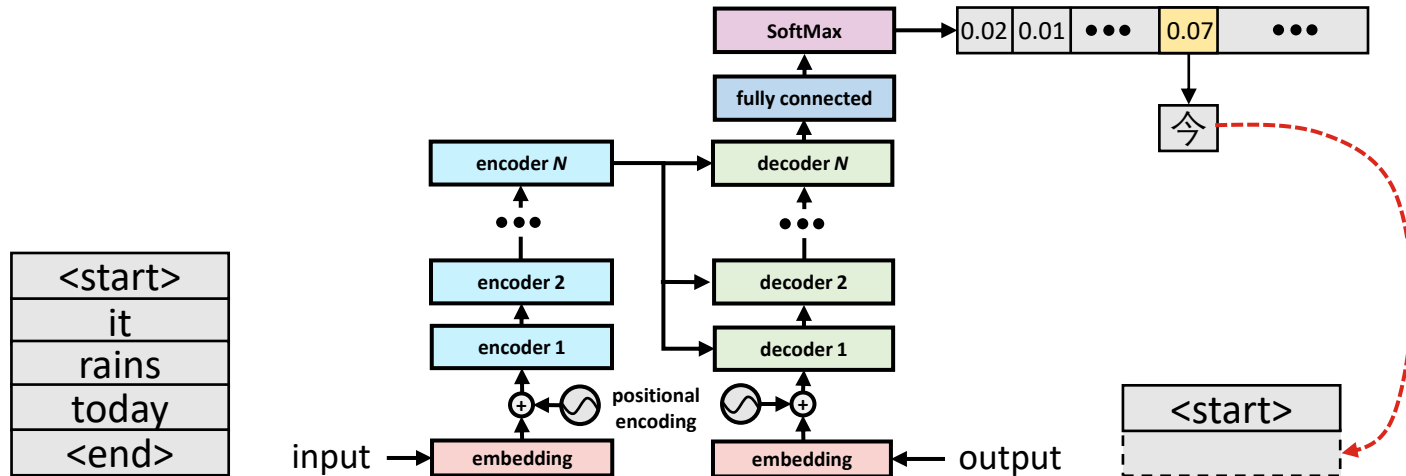
Transformer encoder-decoder in practice

- Model output generated stepwise token by token
 - Output sequence initialised with start symbol only



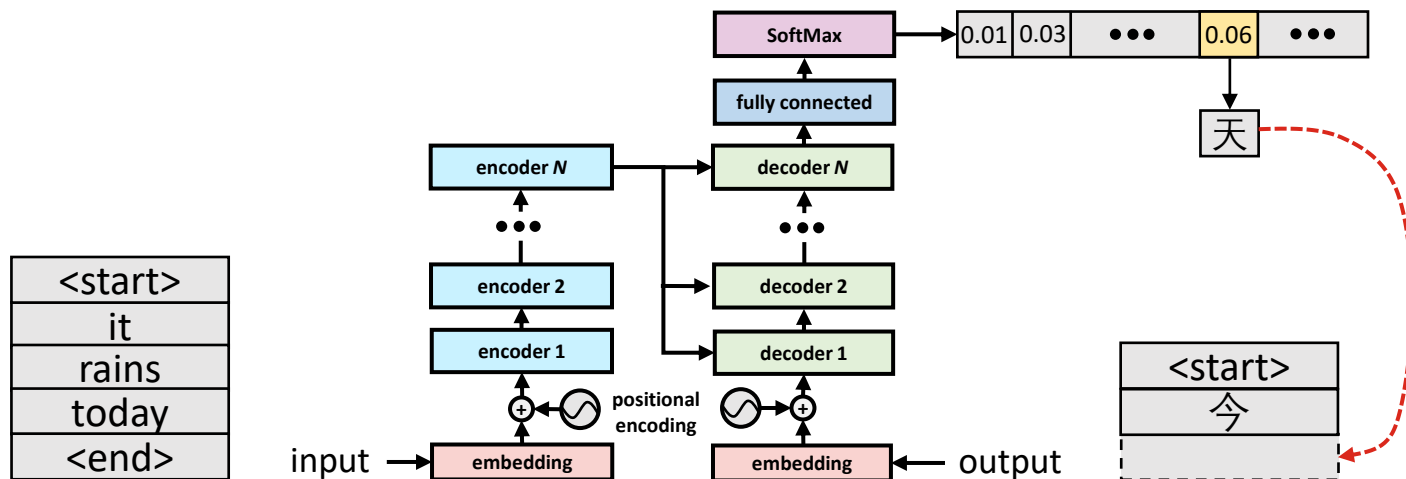
Transformer encoder-decoder in practice

- The first output symbol generated by mapping the output probability vector to the symbol with the highest probability



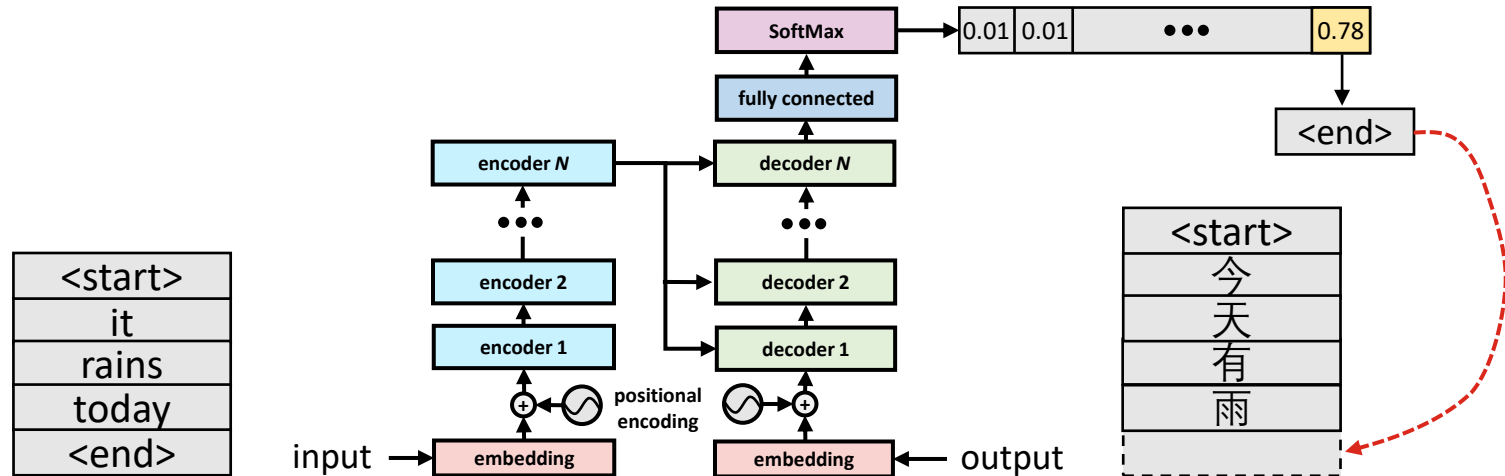
Transformer encoder-decoder in practice

- The output symbol added to the output sequence, and the process is repeated to produce the next output symbol



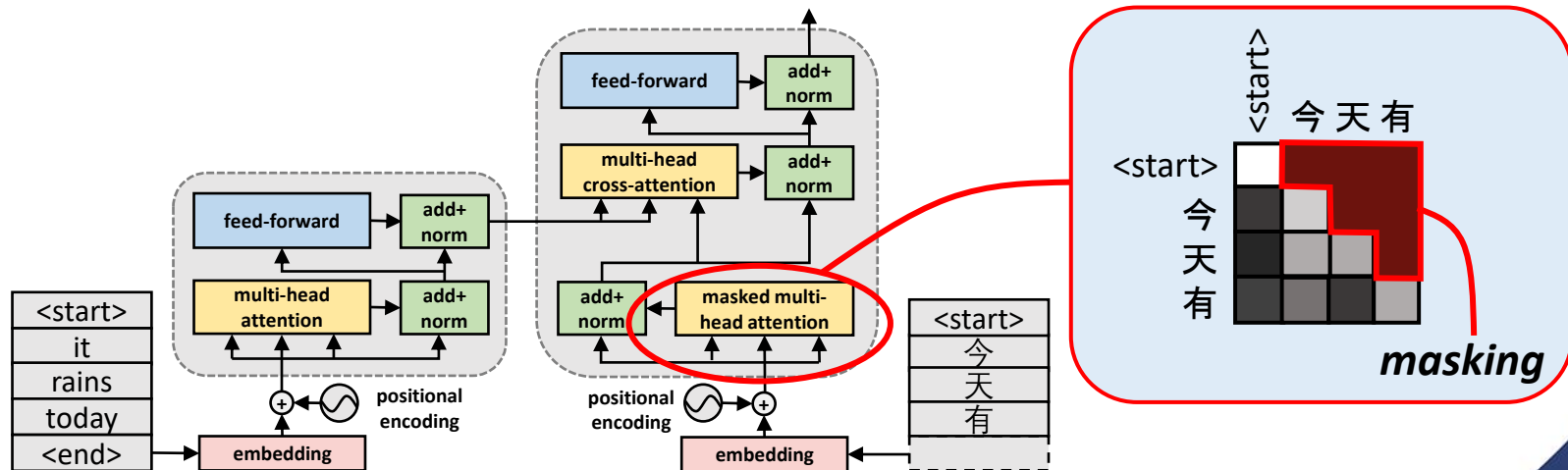
Transformer encoder-decoder in practice

- The process is repeated until the output sequence is ready



Masked attention

- Decoder uses masked attention: output symbols generated later in the sequence are masked by setting their attention weights to zeros



Applications of transformers

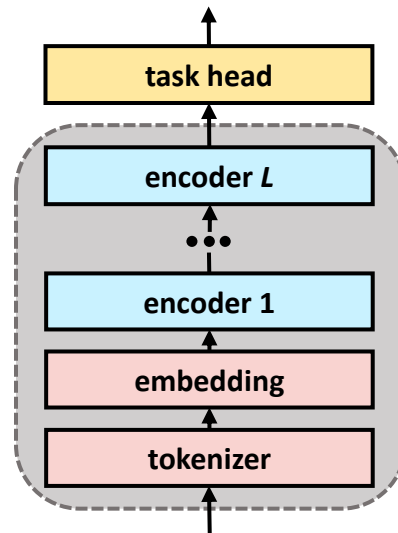
- Transformers have achieved state-of-the-art results in many natural language processing (NLP) applications, for example:
 - **Machine translation** is the most obvious application of transformers
 - **Question answering:** transformers can use their knowledge base to formulate answers to questions in natural language
 - **Automatic summarisation:** transformers can generate a concise summary using a longer text as input
 - **Text classification:** a prediction head can be added to a transformer model to classify text (e.g., user manual, opinion piece, etc.)
 - **Sentiment analysis** is basically a specific case of text classification

Variants of transformers

- The transformer architecture presented above was designed to replace RNN architectures with attention
- Other variants of transformers also exist:
 - *Encoder only transformers* are useful for text classification type of tasks
 - *Decoder only transformers* are useful for text generation without input
 - Different model architectures varying in their technical parameters and learning strategies have been developed
- We will briefly introduce two famous transformers, *BERT* and *GPT*

BERT

- *Bidirectional encoder representations from transformers (BERT)* is a pre-trained encoder-only transformer model for fine-tuning
 - BERT converts input sequence to its latent representation of H -dimensional vector for each input token
 - The model can be fine-tuned for different tasks by replacing the task head
 - Different BERT versions with different number of layers L and hidden size H :
 - BERT_{BASE} with 12L/768H
 - BERT_{LARGE} with 24L/1024H

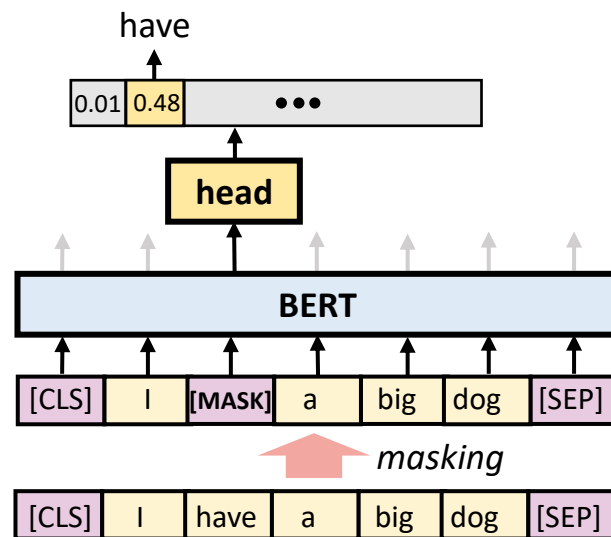


BERT pre-training

- The good performance of BERT is due to self-supervised pre-training not requiring human annotated data
- The pre-training tasks must consider the characteristics of BERT
 - BERT encodes the sequence of input tokens into a sequence of vectors where one vector corresponds to one token
 - BERT cannot be prompted or used to generate text
- BERT models are pre-trained for two self-supervised training tasks: *masked language modeling (MLM)* and *next sentence prediction*

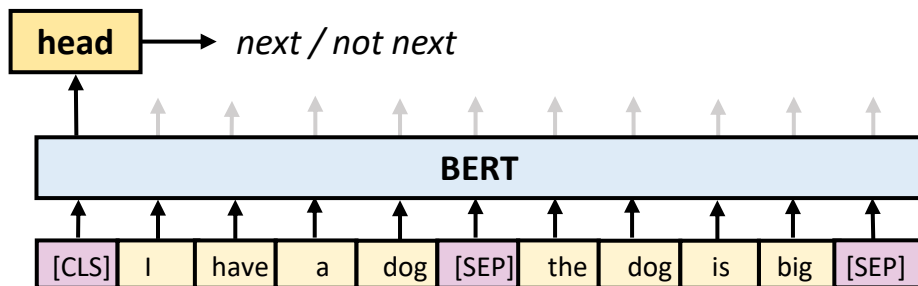
BERT masked language modeling

- In *masked language modeling (MLM)* task, one input token is randomly masked, and the task is to predict the missing token
 - The output vector corresponding to the masked token passed to the prediction head
 - The probability vector representing the probability distribution over the vocabulary space aims to predict the masked token



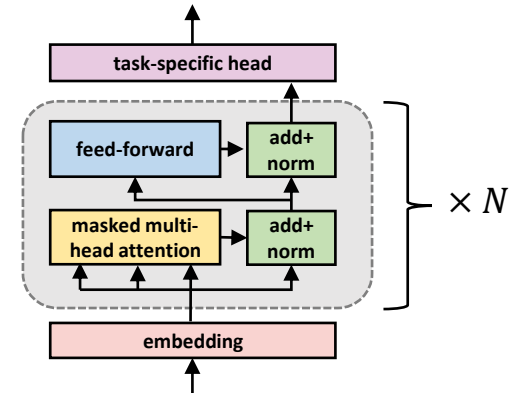
BERT next sequence prediction task

- In *next sequence prediction* task, the input consists of two sentences separated with a special [SEP] token
 - Sentences are chosen randomly so that they either appear sequentially in the text corpus or not
 - The prediction head is trained to perform binary classification task: the second sequence is *next* or *not next* after the first sequence



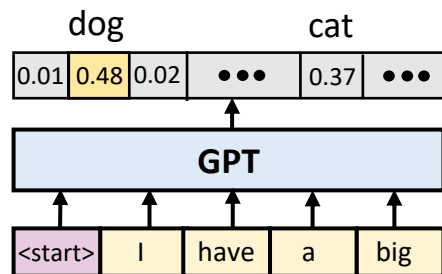
GPT

- Although BERT performs well in tasks such as text classification, it cannot be trained to generate text
- *Generative pre-trained transformer (GPT)* model resembles BERT, but it was designed for text generation rather than encoding
 - GPT is considered as *decoder-only transformer*; as there is no encoder, transformers include no cross-attention layer, only masked MHA layer
 - GPT outputs one token at a time, like transformer encoder-decoders



GPT training

- The original GPT model (GPT-1) was pre-trained using over 7,000 books representing different genres
 - Input tokens define the context; the model is pre-trained to predict the next token in the input sequence
 - In fine-tuning, input sequences can be divided into the context part and the predicted part that the model learns to generate



Pros and cons of transformers

Advantages

- Parallelisation: transformers can be trained efficiently using parallel processing capabilities
- Handling long-term dependencies: attention model concerns long term context better than RNNs
- Scalability and versatility: powerful for many applications

Disadvantages

- High complexity: requires long training time; inference can also be slow
- Large model size: requires a lot of training data, prone to overfit
- Poor interpretability: “black box” models hard for humans to interpret and understand

Summary

- *Attention mechanism* aims to predict the mutual importance of elements in a sequence more accurately than in RNNs
 - *Scaled dot-product attention* predicts mutual importance as attention weights using learnable *query* (Q), *key* (K), and *value* (V) matrices
 - *Self-attention* predicts the weights between tokens in the same sequence
 - *Cross-attention* predicts the weights between tokens in two sequences
- *Transformers* are a family of model architectures based on attention
 - Transformer encoder-decoder models have achieved excellent results in many NLP tasks, such as machine translation and question answering
 - The drawback of transformers is their complexity and large size

Questions and discussion